Detector de Phishing en Correo Electrónico (EN/ES)

Sistema completo de detección de phishing en correos electrónicos en inglés y español, desarrollado como Trabajo de Fin de Grado. Combina un pipeline de procesamiento de lenguaje natural con aprendizaje automático supervisado y una interfaz web accesible sin conocimientos técnicos.

Características principales

- Análisis de texto libre y de archivos adjuntos (`.txt`, `.eml`, `.pdf`, `.docx`)
- Detección automática de idioma (inglés y español)
- Extracción de características explicativas: URLs, dominios, palabras de urgencia, longitud
- Análisis de cabeceras EML: detección de discrepancias entre From y Reply-To
- Tokens más influyentes en la decisión del modelo
- Historial de análisis almacenado en el navegador
- Exportación del informe de análisis a PDF
- Vista comparativa de métricas de los cuatro modelos evaluados
- Mecanismo de feedback para retroalimentación activa del usuario
- Autenticación opcional mediante API key
- Contenedor (Podman) listo para despliegue

Estructura del proyecto

```
├── backend/
│   ├── main.py                 # API FastAPI: endpoints, auth, logs
│   └── ml/
│       ├── build_dataset.py    # Dataset combinado (EN + ES)
│       ├── preprocessing.py    # Pipeline NLP (8 pasos)
│       ├── train.py            # Entrenamiento con Reg. Logística
│       ├── compare_models.py   # Comparativa de los 4 modelos
│       ├── evaluate_spanish.py # Evaluación sobre subset español
│       ├── visualize.py        # Generación de gráficas
│       ├── predict.py          # Predicción y explicación
│       └── phishing_model.joblib  # Modelo entrenado
├── frontend/
│   ├── index.html              # Interfaz web (3 pestañas)
│   ├── styles.css              # Estilos
│   └── app.js                  # Lógica cliente, historial, PDF
├── data/
│   ├── emails.csv              # Dataset combinado
│   ├── resultados_modelos.json # Métricas de los 4 modelos
│   ├── resultados_modelos.csv  # Tabla resumen en CSV
│   ├── manual_es_legit.csv     # Legítimos en español (87)
│   ├── manual_es_phishing.csv  # Phishing en español (56)
│   ├── url_blacklist.txt       # Lista negra de dominios
│   └── feedback.jsonl          # Valoraciones del usuario
├── graficas/
│   ├── 01_comparativa_metricas.png
│   ├── 02_matrices_confusion.png
```

```
│   ├── 03_tiempo_entrenamiento.png
│   └── 04_radar_metricas.png
├── tests/                        # 65 tests (pytest)
│   ├── __init__.py               # Marca tests/ como paquete Python
│   ├── test_api.py               # 24 tests: endpoints
│   ├── test_predict.py           # 20 tests: features, umbrales
│   ├── test_preprocessing.py     # 12 tests: pipeline NLP
│   └── test_eml_parser.py        # 9 tests: parser EML
├── ejemplos/                     # Correos de muestra para demos manuales
│   ├── ejemplo_legitimo.eml
│   ├── ejemplo_phishing.eml
│   ├── legitimo_espanol.txt
│   ├── legitimo_ingles.txt
│   ├── phishing_espanol.txt
│   └── phishing_ingles.txt
├── Dockerfile                    # Imagen de contenedor (Podman)
├── docker-compose.yml            # Orquestación local
├── .dockerignore                 # Exclusiones del build
├── requirements.txt              # Dependencias Python
├── tfg_app.tar                   # Imagen exportada (opcional, generable con
podman save)
└── README.md
```

Dataset

El sistema combina cuatro fuentes de datos para construir el dataset de entrenamiento:

Fuente	Correos	Legítimos	Phishing	Idioma
CEAS_08	39.154	17.312	21.842	Inglés
Enron	29.767	15.791	13.976	Inglés
Manual ES legítimos	87	87	0	Español
Manual ES phishing	56	0	56	Español
Total	**69.064**	**33.190**	**35.874**	

Los archivos `CEAS_08.csv` y `Enron.csv` deben descargarse por separado y colocarse en la raíz del proyecto antes de ejecutar `build_dataset.py`.

Instalación y ejecución en local

1. Crear entorno virtual e instalar dependencias

```
python -m venv .venv
.venv\Scripts\activate        # Windows
# source .venv/bin/activate   # Linux/Mac
pip install -r requirements.txt
```

2. Construir el dataset

Coloca `CEAS_08.csv` y `Enron.csv` en la raíz del proyecto y ejecuta:

```
python -m backend.ml.build_dataset
```

Esto genera `data/emails.csv` con los 69.064 correos combinados.

3. Entrenar y comparar modelos

```
python -m backend.ml.compare_models
```

Este script entrena los cuatro modelos (Regresión Logística, SVM Lineal, Random Forest y Red Neuronal MLP), guarda el mejor en `backend/ml/phishing_model.joblib` y exporta las métricas a `data/resultados_modelos.json`. El modelo ganador es **SVM Lineal** con F1 macro = 0.9898.

4. Arrancar el servidor

Desde la raíz del proyecto, **active el entorno virtual** (si no lo hizo en el paso 1, el prompt debe mostrar `(.venv)`):

```
# Windows (PowerShell)
.venv\Scripts\activate

# Linux / macOS
# source .venv/bin/activate
```

A continuación arranque Uvicorn:

```
uvicorn backend.main:app --reload
```

Si PowerShell indica que `uvicorn` no se reconoce, use una de estas alternativas (no requieren que `uvicorn` esté en el PATH global):

```
# Con el entorno activado
python -m uvicorn backend.main:app --reload

# Windows: sin activar el entorno (ruta explícita al Python del proyecto)
.\.venv\Scripts\python.exe -m uvicorn backend.main:app --reload
```

La aplicación estará disponible en `http://localhost:8000`.

> **Nota (Windows):** si `activate` falla por la política de ejecución, ejecute una vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` en PowerShell o use directamente `.\.venv\Scripts\python.exe -m uvicorn ...`.

Ejecución con Podman

El proyecto usa **Podman** (no Docker Desktop). El `Dockerfile` y `docker-compose.yml` son compatibles con Podman.

```
# Construir la imagen
podman build -t phishing-detector .

# Ejecutar el contenedor
podman run -p 8000:8000 `
  -v "${PWD}/data:/app/data" `
  -v "${PWD}/graficas:/app/graficas" `
  phishing-detector
```

O con orquestación local (desde la raíz del proyecto):

```
podman-compose up --build
```

La aplicación estará disponible en `http://localhost:8000`.

> **Nota:** El modelo entrenado (`phishing_model.joblib`) debe existir antes de construir la imagen, ya que se copia dentro del contenedor. Ejecuta `compare_models.py` antes de `podman build` o `podman-compose up --build`.

Distribución como imagen exportada

Como alternativa al despliegue desde el código fuente, la imagen construida se puede exportar a un único archivo `tfg_app.tar` y entregarse a otro equipo que solo necesita tener Podman instalado. El receptor no requiere Python, `podman-compose` ni el código fuente.

Generación del archivo (en el equipo desarrollador, con la imagen ya construida):

```
podman save -o tfg_app.tar localhost/tfg_app:latest
```

El archivo resultante ocupa aproximadamente 1 GB y contiene la imagen base, las dependencias, los recursos de NLTK, el modelo entrenado y todos los activos estáticos.

Carga y ejecución (en el equipo destino):

```
podman load -i tfg_app.tar
```

```
podman run -d -p 8000:8000 --name tfg_app localhost/tfg_app:latest
```

La aplicación estará disponible en `http://localhost:8000` en segundos, sin descargar dependencias ni construir nada.

Gestión del contenedor:

```
podman ps                 # estado y mapeo de puertos
podman logs tfg_app       # logs del servidor
podman stop tfg_app       # detener
podman rm tfg_app         # eliminar el contenedor
```

Ejecutar los tests

```
pytest tests/ -v
```

La suite contiene 65 tests organizados en cuatro archivos:

- `tests/test_preprocessing.py` (12 tests): pipeline NLP
- `tests/test_eml_parser.py` (9 tests): parser de archivos EML
- `tests/test_predict.py` (20 tests): detección de idioma, extracción de características y umbrales
- `tests/test_api.py` (24 tests): endpoints de la API e integración completa

Endpoints de la API

Método	Endpoint	Descripción	Auth
POST	`/api/predict-text`	Analiza texto libre	Opcional
POST	`/api/predict-file`	Analiza uno o varios archivos	Opcional
GET	`/api/metrics`	Métricas comparativas de los modelos	No
POST	`/api/feedback`	Registra valoración del usuario	Opcional

La documentación interactiva de la API está disponible en `http://localhost:8000/docs`.

Autenticación por API key (opcional)

Por defecto el sistema arranca en **modo abierto** (sin definir `API_KEYS`). Es la configuración recomendada para la **interfaz web** del TFG.

Para restringir el acceso a la API, define la variable de entorno `API_KEYS` con una o más claves separadas por comas. En ese caso:

Endpoint	¿Requiere cabecera `X-API-Key`?
`POST /api/predict-text`	Sí
`POST /api/predict-file`	Sí
`POST /api/feedback`	Sí
`GET /api/metrics`	No

Si la clave falta o no es válida, la API responde **401** (*API key inválida o ausente*).

Importante: el frontend **no** envía la cabecera `X-API-Key`. Si defines `API_KEYS`, el análisis desde el navegador fallará con error 401. Usa la autenticación para clientes externos (Postman, `/docs`) o integraciones; para la web, deja `API_KEYS` sin configurar.

Windows (PowerShell)

Antes de arrancar el servidor (véase el paso 4 de instalación local):

```
$env:API_KEYS = "clave-a,clave-b"
.\.venv\Scripts\python.exe -m uvicorn backend.main:app --reload --host 127.0.0.1 -
-port 8000
```

Para volver al modo abierto en la misma sesión:

```
Remove-Item Env:API_KEYS
```

Podman

```
podman run -p 8000:8000 -e API_KEYS="clave-a" phishing-detector
```

Con `docker-compose.yml` y **podman-compose**, descomenta y ajusta en el servicio `app`:

```
environment:
  - API_KEYS=clave-a,clave-b
```

Luego reinicia: `podman-compose up -d`

Ejemplo de petición autenticada (PowerShell)

```
Invoke-RestMethod -Uri "http://localhost:8000/api/predict-text" -Method POST `
  -Headers @{ "Content-Type" = "application/json"; "X-API-Key" = "clave-a" } `
```

```
    -Body '{"text": "Please verify your account immediately"}'
```

La documentación interactiva en `http://localhost:8000/docs` permite probar los endpoints con el botón **Authorize**.